

Job Card Management System - Frontend Technical Documentation

Developer: Solo Project

Technology Stack: React 18, React Router, Tailwind CSS, Context API

Status: Development (with mock data)

Presentation Duration: 1-2 hours

Audience: Senior Software Engineers (Technical Review Panel)

Table of Contents

1. [Frontend Overview](#)
 2. [Architecture & Design Patterns](#)
 3. [Component Structure](#)
 4. [State Management](#)
 5. [Routing & Navigation](#)
 6. [Authentication Flow](#)
 7. [API Integration](#)
 8. [UI/UX Design System](#)
 9. [Key Features Implementation](#)
 10. [Performance Optimizations](#)
 11. [Code Walkthrough](#)
 12. [Build & Deployment](#)
 13. [Future Enhancements](#)
-

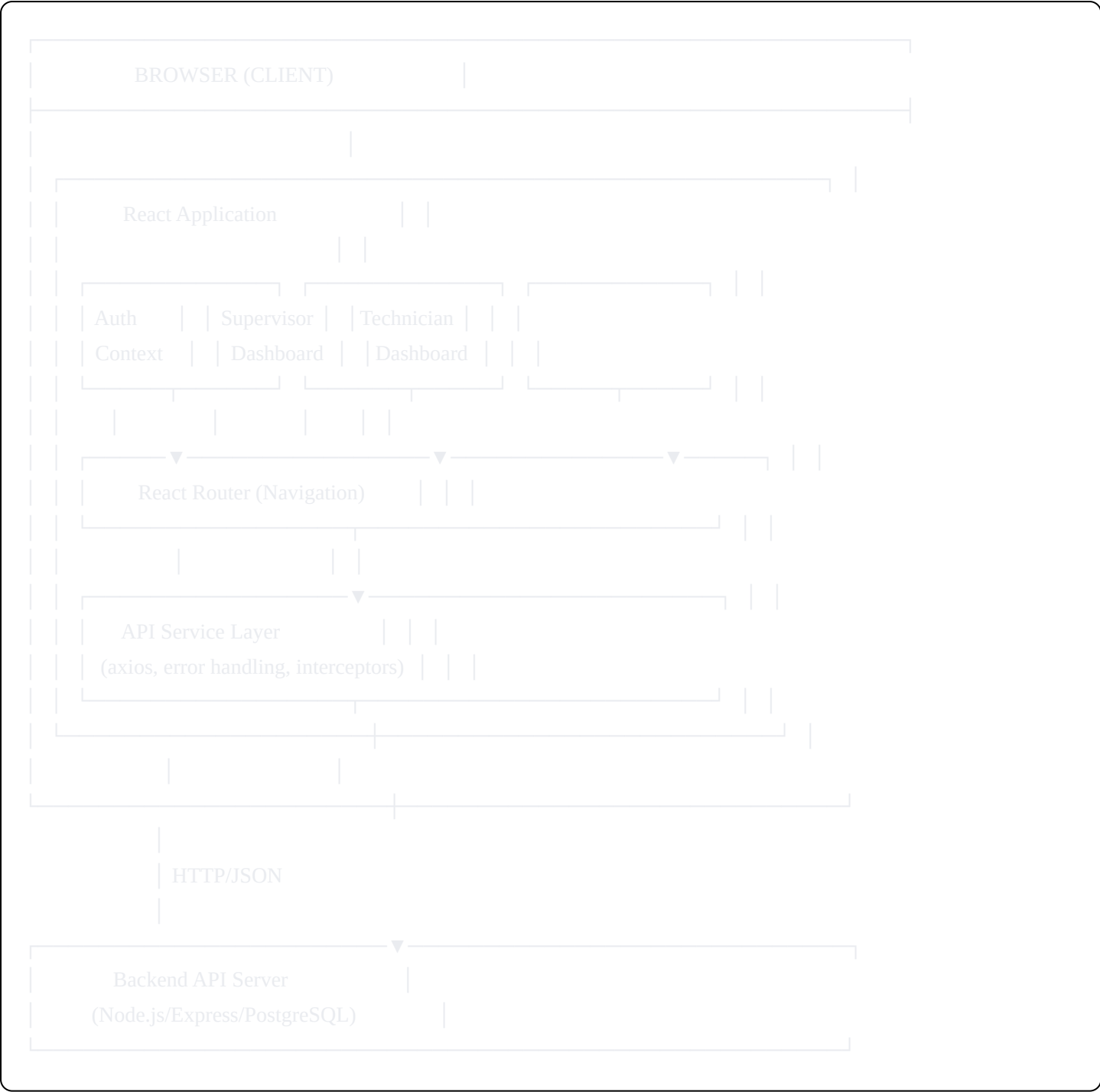
1. Frontend Overview

1.1 Problem Statement (Frontend Perspective)

The frontend must provide:

- **Role-specific interfaces:** Different UX for technicians vs supervisors
- **Real-time updates:** Job status changes reflected immediately
- **Mobile-friendly:** Technicians use phones in the field
- **Offline-capable:** Work even with poor connectivity (future)
- **Intuitive UX:** Minimal training required

1.2 Solution Architecture



1.3 Tech Stack Justification

Technology	Reason for Selection
React 18	Component reusability; large ecosystem; virtual DOM performance
React Router v6	Declarative routing; nested routes; built-in hooks
Context API	Global state (auth) without Redux overhead
Tailwind CSS	Utility-first; rapid prototyping; consistent design
Custom Fonts	Syne (headings) + DM Sans (body) for brand identity

Technology	Reason for Selection
Vite	Fast dev server; HMR; optimized builds

Why NOT Redux?

- **Small state surface:** Only auth + fetched data
- **Server as source of truth:** Most state lives in backend
- **Simpler mental model:** Context API sufficient for this scale
- **Trade-off:** If app grows, Redux/Zustand could be added

2. Architecture & Design Patterns

2.1 Component Architecture

```
src/
├── components/
│   ├── auth/          # Authentication guards
│   │   └── ProtectedRoute.jsx
│   ├── ui/            # Reusable UI components
│   │   ├── EmptyState.jsx
│   │   ├── Logo.jsx
│   │   ├── Skeleton.jsx
│   │   └── Toast.jsx
│   └── [common/, layout/] # (Future: shared components)
├── pages/
│   ├── auth/          # Public pages
│   │   ├── LoginPage.jsx
│   │   └── RegisterPage.jsx
│   ├── supervisor/    # Supervisor-only pages
│   │   ├── SupervisorDashboard.jsx # Layout + routing
│   │   ├── StatsOverview.jsx
│   │   ├── AllJobsList.jsx
│   │   ├── CreateJobCard.jsx
│   │   ├── SupervisorJobDetail.jsx
│   │   ├── CustomerList.jsx
│   │   ├── UserList.jsx
│   │   └── UserDetail.jsx
│   └── technician/    # Technician-only pages
│       ├── TechnicianDashboard.jsx # Layout + routing
│       ├── JobCardList.jsx
│       └── JobDetail.jsx
```

```
|— context/
|   |— AuthContext.jsx # Global auth state
|
|— services/
|   |— api.js # API integration layer
|
|— App.jsx # Root component + routing
```

2.2 Design Patterns Used

2.2.1 Container/Presentational Pattern

```
javascript

// Container (handles logic)
const JobCardList = () => {
  const [jobs, setJobs] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    jobCardAPI.getAll().then(setJobs).finally(() => setLoading(false));
  }, []);

  if (loading) return <SkeletonJobList />;
  if (jobs.length === 0) return <EmptyState type="jobs" />;

  return jobs.map(job => <JobCard key={job.id} job={job} />);
};

// Presentational (pure display)
const JobCard = ({ job }) => (
  <div className="...">
    <h3>{job.title}</h3>
    <StatusBadge status={job.status} />
  </div>
);
```

Benefits:

- **Reusability:** `JobCard` can be used anywhere
- **Testability:** Pure components easy to test
- **Separation:** Data fetching $\neq$ rendering

2.2.2 Render Props Pattern (Toast)

```
javascript
```

```
// Toast provider wraps app
<ToastProvider>
  {children}
  { /* Toast container rendered here */ }
</ToastProvider>

// Consumer accesses via hook
const { toast } = useToast();
toast.success('Job completed!');
```

2.2.3 Higher-Order Component (Protected Route)

```
javascript

// Wraps routes that require authentication
<Route path="/supervisor/*" element={
  <ProtectedRoute requiredRole="supervisor">
    <SupervisorDashboard />
  </ProtectedRoute>
} />
```

Why HOC here?

- Declarative route protection
- Centralized auth logic
- Role-based redirects

2.2.4 Custom Hooks Pattern

```
javascript

// services/api.js exports API functions
export const jobCardAPI = {
  getAll: (filters) => fetchWithAuth('/job-cards', { params: filters }),
  getById: (id) => fetchWithAuth(`/job-cards/${id}`),
  // ...
};

// Components use directly (no custom hook wrapper needed yet)
const jobs = await jobCardAPI.getAll({ status: 'pending' });
```

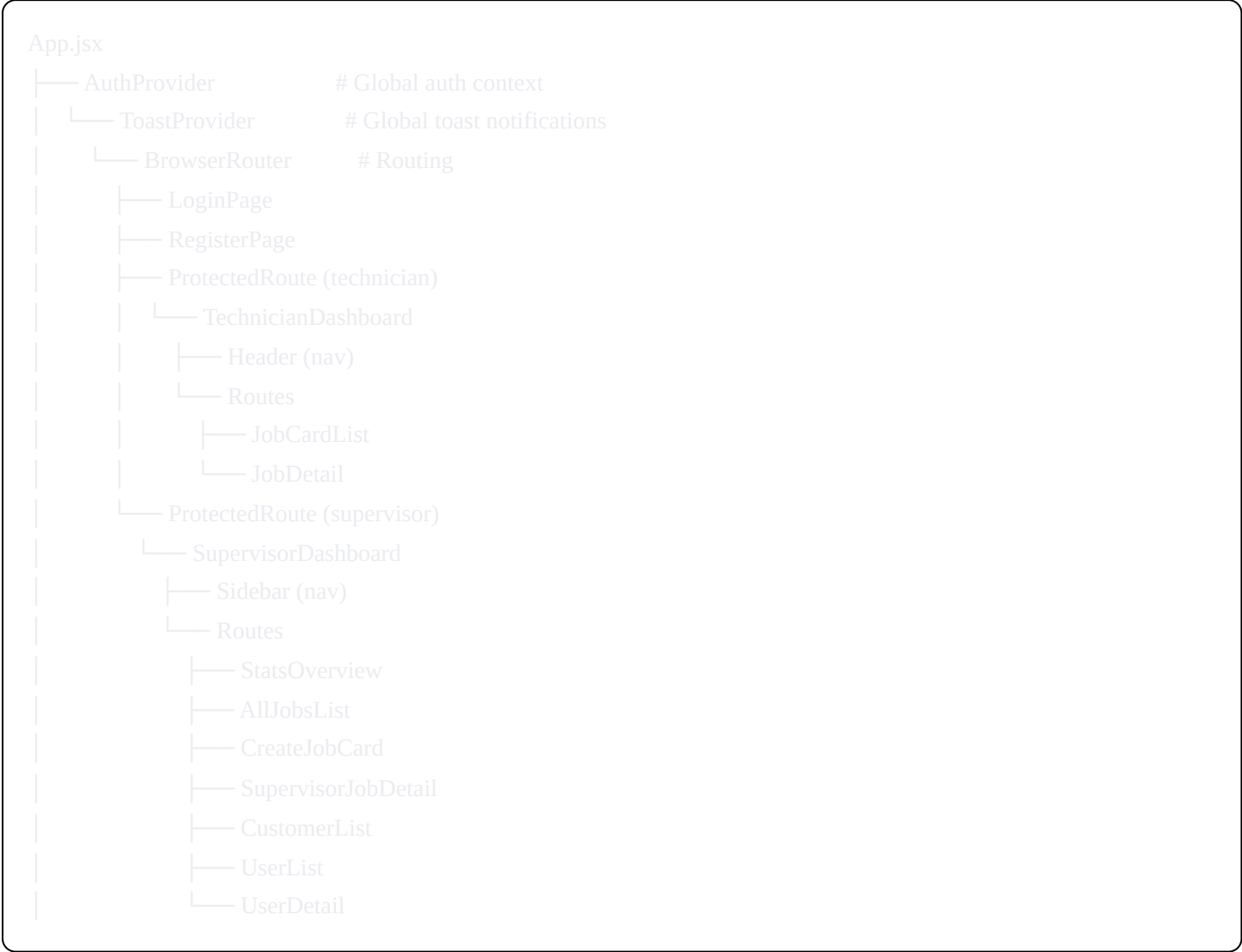
Why NOT custom hooks like `useJobCards()`?

- **Current scope:** Simple data fetching
- **Server as truth:** No complex client-side state

- **Trade-off:** Could add `react-query` for caching/optimistic updates
-

3. Component Structure

3.1 Component Hierarchy



3.2 Shared UI Components

3.2.1 EmptyState Component

javascript

```
// components/ui/EmptyState.jsx
const EmptyState = ({
  type = 'jobs', // Predefined types: jobs, customers, users, search, completed
  title, // Optional custom title
  description, // Optional custom description
  onAction, // Optional action button handler
  actionLabel, // Action button text
  query, // Search query (for search empty state)
}) => {
  const defaults = {
    jobs: { title: 'No job cards yet', description: 'Create your first job card...' },
    customers: { title: 'No customers yet', description: 'Add your first customer...' },
    // ...
  };

  const config = defaults[type] || defaults.jobs;

  return (
    <div className="flex flex-col items-center justify-center py-16 px-6 text-center">
      <div className="w-32 h-28 mb-6">
        {illustrations[type] || illustrations.jobs}
      </div>
      <h3 className="text-base font-semibold text-gray-800 mb-1.5">
        {title || config.title}
      </h3>
      <p className="text-sm text-gray-400 max-w-xs leading-relaxed mb-6">
        {description || config.description}
      </p>
      {onAction && actionLabel && (
        <button onClick={onAction} className="...">
          {actionLabel}
        </button>
      )}
    </div>
  );
};
```

Design Decisions:

1. **Predefined types:** Common cases have defaults (DRY principle)
2. **Override capability:** Custom title/description for edge cases
3. **Inline SVG illustrations:** No external image dependencies
4. **Optional action:** Not all empty states need actions

Usage Examples:

javascript

// Simple (uses defaults)

```
<EmptyState type="jobs" />
```

// With action

```
<EmptyState  
  type="jobs"  
  onAction={() => navigate('/supervisor/jobs/new')}  
  actionLabel="Create Job Card"  
>
```

// Search results

```
<EmptyState  
  type="search"  
  query={searchTerm}  
  description="Try different keywords or filters"  
>
```

3.2.2 Skeleton Component

javascript

```
// components/ui/Skeleton.jsx
```

```
// Base shimmer effect
```

```
export const Skeleton = ({ className = "" }) => (  
  <>  
    <style>{`  
      @keyframes shimmer {  
        0% { background-position: -600px 0; }  
        100% { background-position: 600px 0; }  
      }  
      .skeleton-shimmer {  
        background: linear-gradient(90deg,  
          #f0f0f0 25%, #e0e0e0 37%, #f0f0f0 63%);  
        background-size: 600px 100%;  
        animation: shimmer 1.4s ease-in-out infinite;  
      }  
    `}</style>  
    <div className={`skeleton-shimmer ${className}`} />  
  </>  
);
```

```
// Skeleton for job card
```

```
export const SkeletonJobCard = () => (  
  <div className="bg-white rounded-xl border border-gray-100 p-4">  
    <div className="flex items-start justify-between gap-3">  
      <div className="flex-1 space-y-2">  
        <Skeleton className="h-5 w-20" />  
        <Skeleton className="h-4 w-3/4" />  
        <Skeleton className="h-3 w-1/2" />  
      </div>  
      <Skeleton className="h-4 w-16" />  
    </div>  
  </div>  
);
```

```
// Skeleton for entire list
```

```
export const SkeletonJobList = ({ count = 4 }) => (  
  <div className="space-y-3">  
    {Array.from({ length: count }).map((_, i) => (  
      <SkeletonJobCard key={i} />  
    ))}  
  </div>  
);
```

```
// Skeleton for table
```

```
export const SkeletonTable = ({ rows = 5, cols = 6 }) => (  
  <div>
```

```

<div className="bg-white rounded-xl border border-gray-100 overflow-hidden">
  <table className="w-full">
    <thead>
      <tr className="bg-gray-50 border-b border-gray-100">
        {Array.from({ length: cols }).map((_, i) => (
          <th key={i} className="px-4 py-3">
            <Skeleton className="h-3 w-16" />
          </th>
        ))}
      </tr>
    </thead>
    <tbody>
      {Array.from({ length: rows }).map((_, i) => (
        <tr key={i}>
          {Array.from({ length: cols }).map((_, j) => (
            <td key={j} className="px-4 py-3.5">
              <Skeleton className="h-3.5 w-24" />
            </td>
          ))}
        </tr>
      ))}
    </tbody>
  </table>
</div>
);

```

Why Skeleton Screens?

1. **Perceived performance:** User sees something immediately
2. **Layout stability:** No content shift when data loads
3. **Better UX:** Skeleton hints at content structure
4. **Alternative to spinners:** Less jarring than spinner → content jump

Performance Note:

CSS animations are GPU-accelerated (smooth on mobile).

3.2.3 Toast Notification System

javascript

```

// components/ui/Toast.jsx

// Provider manages toast state
export const ToastProvider = ({ children }) => {
  const [toasts, setToasts] = useState([]);
  let toastId = 0;

  const dismiss = useCallback((id) => {
    setToasts(prev => prev.map(t =>
      t.id === id ? { ...t, leaving: true } : t
    ));
    setTimeout(() =>
      setToasts(prev => prev.filter(t => t.id !== id)),
      350 // Animation duration
    );
  }, []);

  const add = useCallback((message, type = 'info', duration = 4000) => {
    const id = ++toastId;
    setToasts(prev => [...prev, { id, message, type, leaving: false }]);
    if (duration > 0) setTimeout(() => dismiss(id), duration);
    return id;
  }, [dismiss]);

  const toast = {
    success: (msg, dur) => add(msg, 'success', dur),
    error: (msg, dur) => add(msg, 'error', dur ?? 6000),
    info: (msg, dur) => add(msg, 'info', dur),
    warning: (msg, dur) => add(msg, 'warning', dur),
  };

  return (
    <ToastContext.Provider value={{ toast }}>
      {children}
      <div className="fixed top-4 right-4 z-[9999] flex flex-col gap-2">
        {toasts.map(t => (
          <ToastItem key={t.id} toast={t} onDismiss={() => dismiss(t.id)} />
        ))}
      </div>
    </ToastContext.Provider>
  );
};

// Toast item (individual notification)
const ToastItem = ({ toast: t, onDismiss }) => {
  const style = TOAST_STYLES[t.type] || TOAST_STYLES.info;

```

```

return (
  <div style={{
    animation: t.leaving
      ? 'toastOut 0.35s cubic-bezier(0.4,0,1,1) forwards'
      : 'toastIn 0.35s cubic-bezier(0,0,0.2,1) forwards',
  }}>
    <div className="bg-white rounded-xl shadow-xl border border-gray-100 p-4">
      <div className={`absolute left-0 top-0 bottom-0 w-1 ${style.bar}`} />
      <div className="flex items-start gap-3">
        <svg className={`h-5 w-5 ${style.icon}`}>{style.svg}</svg>
        <div className="flex-1">
          <p className="text-xs font-semibold text-gray-400 uppercase">
            {style.label}
          </p>
          <p className="text-sm font-medium text-gray-800">{t.message}</p>
        </div>
        <button onClick={onDismiss}>×</button>
      </div>
    </div>
  </div>
);
};

// Custom hook for easy access
export const useToast = () => {
  const ctx = useContext(ToastContext);
  if (!ctx) throw new Error('useToast must be used within ToastProvider');
  return ctx;
};

```

Usage:

```

javascript

const { toast } = useToast();

toast.success('Job completed successfully!');
toast.error('Failed to save job card');
toast.warning('Connection unstable');
toast.info('New job assigned to you');

```

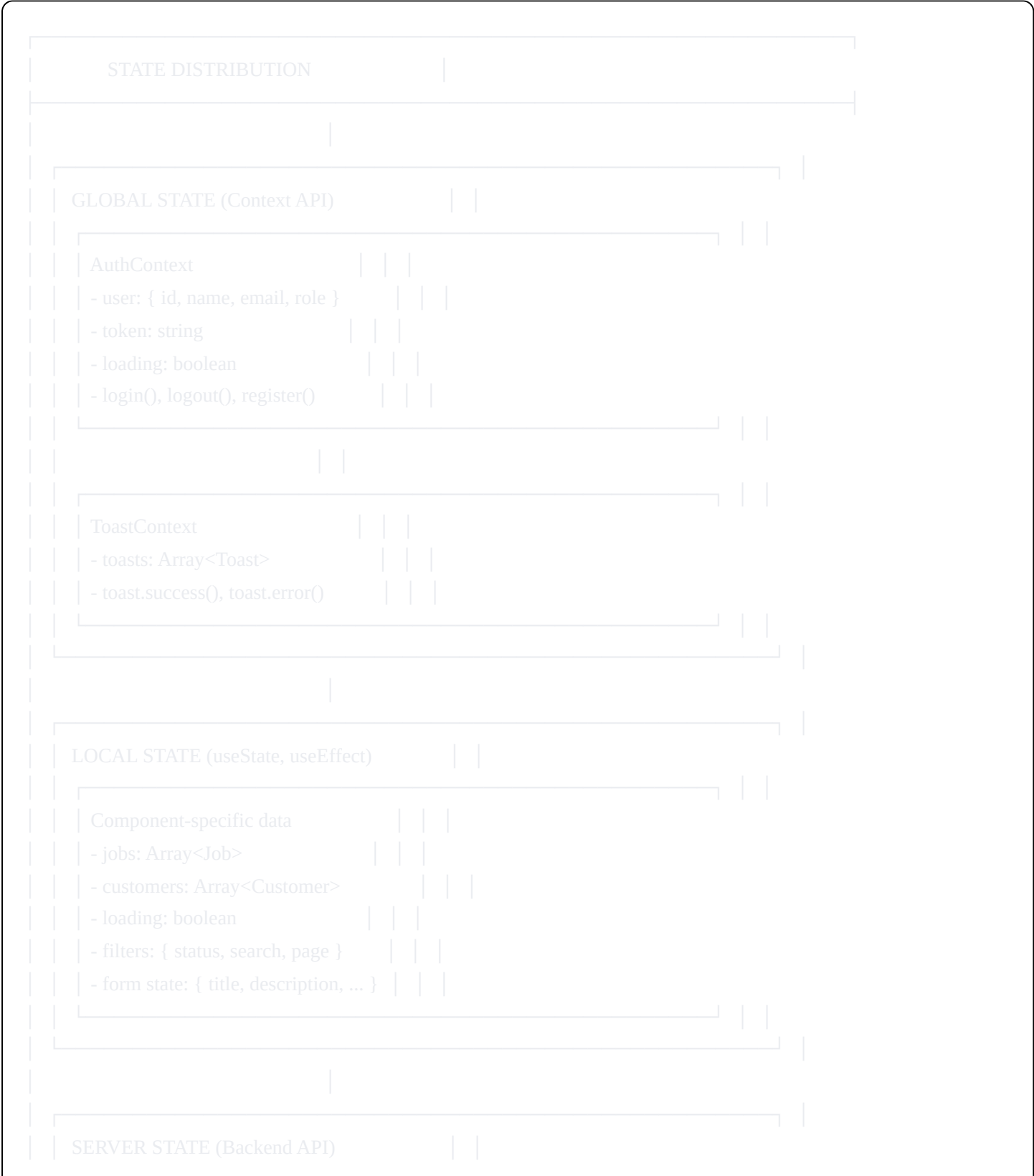
Design Decisions:

1. **Auto-dismiss:** Success toasts disappear after 4s, errors after 6s
2. **Stacking:** Multiple toasts stack vertically (top-right)

- 3. **Animations:** Slide-in from right, slide-out on dismiss
- 4. **Type-based styling:** Color-coded by type (success=green, error=red)
- 5. **Dismissible:** User can manually close
- 6. **z-index 9999:** Above all other content

4. State Management

4.1 State Architecture



- Source of truth for all entity data
- Fetched on demand, cached in local state

4.2 Authentication Context

javascript

```
// context/AuthContext.jsx

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // INITIALIZATION - Load user from localStorage on mount
  useEffect(() => {
    const initializeAuth = () => {
      try {
        const storedUser = localStorage.getItem('user');
        const token = localStorage.getItem('token');

        if (storedUser && token) {
          setUser(JSON.parse(storedUser));
        }
      } catch (error) {
        console.error('Error loading user from localStorage:', error);
        localStorage.removeItem('user');
        localStorage.removeItem('token');
      } finally {
        setLoading(false);
      }
    };

    initializeAuth();
  }, []);

  // LOGIN
  const login = async (email, password) => {
    try {
      const response = await authAPI.login(email, password);
      const { token, user: userData } = response.data;

      if (!token || !userData) {
        throw new Error('Invalid response from server');
      }

      localStorage.setItem('token', token);
      localStorage.setItem('user', JSON.stringify(userData));
      setUser(userData);

      return { success: true, user: userData };
    } catch (error) {
      return {
        success: false,
      };
    }
  };
};
```

```

      error: typeof error === 'string' ? error : error.message
    };
  }
};

// LOGOUT
const logout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  setUser(null);
};

// REGISTER
const register = async (userData) => {
  try {
    await authAPI.register(userData);

    // Auto-login after registration
    const loginResult = await authAPI.login(userData.email, userData.password);
    const { token, user: newUser } = loginResult.data;

    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(newUser));
    setUser(newUser);

    return { success: true, user: newUser };
  } catch (error) {
    return {
      success: false,
      error: typeof error === 'string' ? error : error.message
    };
  }
};

const value = {
  user,
  loading,
  login,
  logout,
  register,
  isAuthenticated: !!user && !!localStorage.getItem('token'),
  isSupervisor: user?.role === 'supervisor',
  isTechnician: user?.role === 'technician',
};

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

```

```
// Custom hook
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within AuthProvider');
  return context;
};
```

Key Design Decisions:

1. **localStorage persistence:** User stays logged in across page refreshes
2. **Auto-initialize on mount:** Check for existing session
3. **Error handling:** Returns `{ success, error }` instead of throwing
4. **Auto-login after register:** Better UX (no manual login needed)
5. **Helper flags:** `isSupervisor`, `isTechnician` for conditional rendering

Security Considerations:

- **localStorage risks:** Vulnerable to XSS (acceptable trade-off for SPA)
- **Token expiry:** Frontend doesn't check expiry (backend enforces)
- **HTTPS required:** localStorage secure only over HTTPS

4.3 Component State Pattern

javascript

```
// Typical component state structure
const JobCardList = () => {
  // 1. Data state
  const [jobs, setJobs] = useState([]);

  // 2. UI state
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // 3. Filter state
  const [status, setStatus] = useState('all');
  const [search, setSearch] = useState("");
  const [page, setPage] = useState(1);

  // 4. Pagination state
  const [pagination, setPagination] = useState(null);

  // Data fetching
  useEffect(() => {
    const fetchJobs = async () => {
      setLoading(true);
      try {
        const filters = { page, limit: 20 };
        if (status !== 'all') filters.status = status;
        if (search.trim()) filters.search = search.trim();

        const res = await jobCardAPI.getAll(filters);
        setJobs(res.data || []);
        setPagination(res.pagination);
      } catch (e) {
        setError(typeof e === 'string' ? e : 'Failed to load jobs');
      } finally {
        setLoading(false);
      }
    };

    fetchJobs();
  }, [page, status, search]); // Re-fetch when filters change

  // Reset page when filters change
  useEffect(() => {
    setPage(1);
  }, [search, status]);
```

```
// Render logic...
```

```
};
```

State Organization:

1. **Data state:** Fetched from API (`jobs`, `customers`, `users`)
2. **UI state:** Loading, errors, modal visibility
3. **Filter state:** User-controlled filters (search, status, date range)
4. **Pagination state:** Current page, total pages, total count

Why `useEffect` for fetching?

- **Automatic:** Runs on mount and when dependencies change
- **Cleanup:** Can return cleanup function if needed
- **Dependencies:** Re-fetches when filters change

5. Routing & Navigation

5.1 Route Structure

```
javascript
```

```
// App.jsx
function App() {
  return (
    <AuthProvider>
    <ToastProvider>
    <BrowserRouter>
    <Routes>
      {/* Public routes */}
      <Route path="/login" element={ <LoginPage /> } />
      <Route path="/register" element={ <RegisterPage /> } />

      {/* Technician routes */}
      <Route path="/technician/*" element={
        <ProtectedRoute requiredRole="technician">
          <TechnicianDashboard />
        </ProtectedRoute>
      } />

      {/* Supervisor routes */}
      <Route path="/supervisor/*" element={
        <ProtectedRoute requiredRole="supervisor">
          <SupervisorDashboard />
        </ProtectedRoute>
      } />

      {/* Default redirect */}
      <Route path="/" element={ <Navigate to="/login" replace /> } />

      {/* 404 */}
      <Route path="*" element={ <NotFoundPage /> } />
    </Routes>
    </BrowserRouter>
    </ToastProvider>
    </AuthProvider>
  );
}
```

5.2 Protected Route Implementation

javascript

```
// components/auth/ProtectedRoute.jsx

const ProtectedRoute = ({ children, requiredRole }) => {
  const { user, loading } = useAuth();

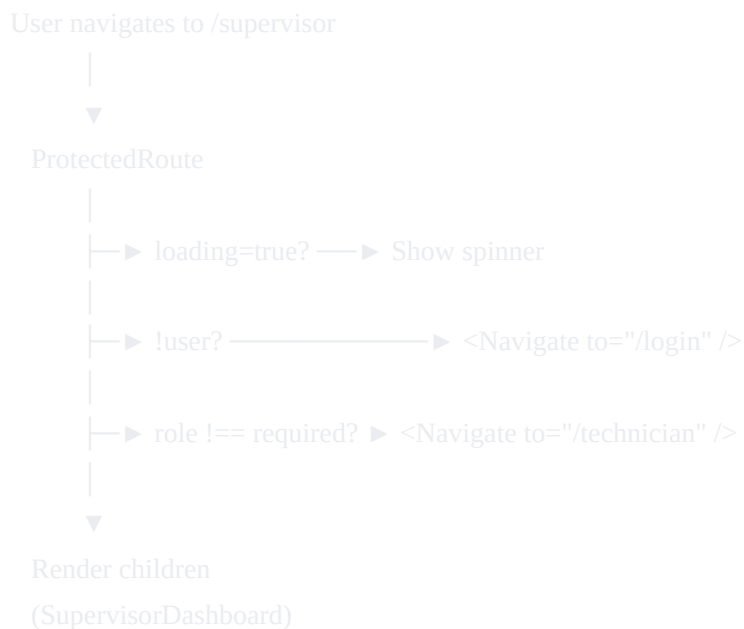
  // While checking authentication, show loading
  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600" />
      </div>
    );
  }

  // No user or token - redirect to login
  if (!user || !localStorage.getItem('token')) {
    return <Navigate to="/login" replace />;
  }

  // Role mismatch - redirect to correct dashboard
  if (requiredRole && user.role !== requiredRole) {
    const redirectPath = user.role === 'supervisor' ? '/supervisor' : '/technician';
    return <Navigate to={redirectPath} replace />;
  }

  // Authorized - render protected content
  return children;
};
```

Flow Diagram:



Why `replace` prop?

- Prevents back button going to protected route when logged out
- Better UX (no redirect loop)

5.3 Nested Routing (Dashboards)

```
javascript

// pages/supervisor/SupervisorDashboard.jsx

const SupervisorDashboard = () => {
  return (
    <div className="flex">
      {/* Sidebar (always visible) */}
      <Sidebar />

      {/* Main content area */}
      <main className="flex-1 p-6">
        <Routes>
          <Route index element={<StatsOverview />} />
          <Route path="jobs" element={<AllJobsList />} />
          <Route path="jobs/new" element={<CreateJobCard />} />
          <Route path="jobs/:id" element={<SupervisorJobDetail />} />
          <Route path="customers" element={<CustomerList />} />
          <Route path="users" element={<UserList />} />
          <Route path="users/:id" element={<UserDetail />} />
        </Routes>
      </main>
    </div>
  );
};
```

URL Structure:

```
/supervisor      → StatsOverview
/supervisor/jobs  → AllJobsList
/supervisor/jobs/new → CreateJobCard
/supervisor/jobs/42 → SupervisorJobDetail (id=42)
/supervisor/customers → CustomerList
/supervisor/users → UserList
/supervisor/users/5 → UserDetail (id=5)
```

Why Nested Routes?

1. **Layout persistence:** Sidebar stays mounted

2. **Code splitting:** Can lazy-load sub-routes
3. **Cleaner URLs:** No `/supervisor-jobs`, just `/supervisor/jobs`

5.4 Programmatic Navigation

```
javascript

import { useNavigate, useLocation } from 'react-router-dom';

const navigate = useNavigate();
const location = useLocation();

// Navigate with state (pass data between routes)
navigate('/supervisor/jobs', {
  state: { statusFilter: 'pending' }
});

// Access state in destination component
const location = useLocation();
const initialStatus = location.state?.statusFilter || 'all';

// Navigate back
navigate(-1);

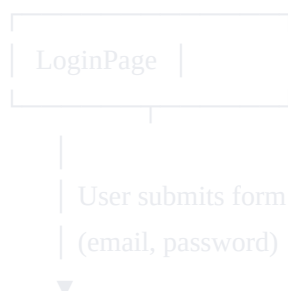
// Replace (don't add to history)
navigate('/login', { replace: true });
```

Use Cases:

- **Filtering:** Click stat card → navigate to jobs list with filter
- **Form submission:** Create job → navigate to job detail
- **Cancel action:** Navigate back to previous page

6. Authentication Flow

6.1 Login Flow (Complete)



Validate inputs

(client-side)

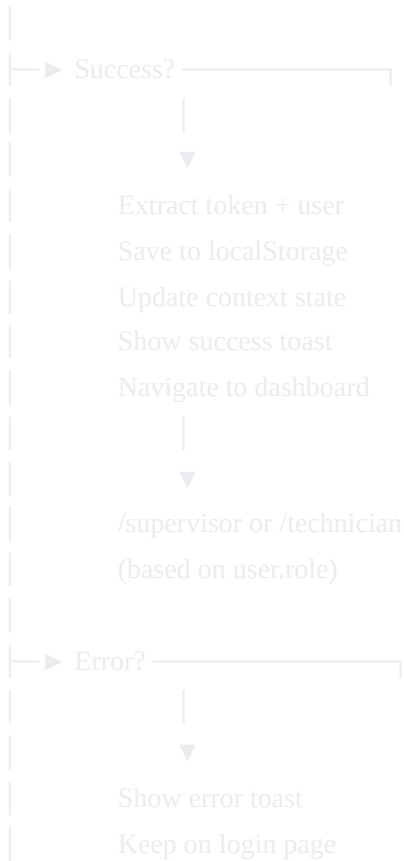


Call login(email, password)

(AuthContext)

POST /api/v1/auth/login

(API service)



[End]

6.2 Login Component Implementation

javascript

```
// pages/auth/LoginPage.jsx
```

```
const LoginPage = () => {  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  const [loading, setLoading] = useState(false);  
  const [emailErr, setEmailErr] = useState("");  
  const [pwdErr, setPwdErr] = useState("");  
  
  const { login } = useAuth();  
  const navigate = useNavigate();  
  const { toast } = useToast();  
  
  // Client-side validation  
  const validate = () => {  
    let ok = true;  
    setEmailErr("");  
    setPwdErr("");  
  
    if (!email.trim()) {  
      setEmailErr('Email is required');  
      ok = false;  
    } else if (!/^[^s@]+@[^s@]+\.[^s@]+$/.test(email)) {  
      setEmailErr('Enter a valid email');  
      ok = false;  
    }  
  
    if (!password) {  
      setPwdErr('Password is required');  
      ok = false;  
    } else if (password.length < 6) {  
      setPwdErr('At least 6 characters');  
      ok = false;  
    }  
  
    return ok;  
  };  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    if (!validate()) return;  
  
    setLoading(true);  
    const result = await login(email, password);  
  
    if (result.success) {
```

```

toast.success(`Welcome back, ${result.user.name.split(' ')[0]}!`);
navigate(result.user.role === 'supervisor' ? '/supervisor' : '/technician');
} else {
  toast.error(result.error || 'Invalid email or password');
}

setLoading(false);
};

return (
  <div className="min-h-screen flex">
    {/* Left panel - branding */}
    <div className="hidden lg:flex lg:w-[42%] bg-slate-900">
      {/* Logo, company info, features */}
    </div>

    {/* Right panel - form */}
    <div className="flex-1 flex items-center justify-center px-6 py-12">
      <form onSubmit={handleSubmit} className="w-full max-w-sm">
        {/* Email field */}
        <input
          type="email"
          value={email}
          onChange={e => { setEmail(e.target.value); setEmailErr(""); }}
          placeholder="you@copycatgroup.com"
          className={emailErr ? 'border-red-400' : ''}
        />
        {emailErr && <p className="text-xs text-red-500">{emailErr}</p>}

        {/* Password field */}
        <input
          type="password"
          value={password}
          onChange={e => { setPassword(e.target.value); setPwdErr(""); }}
          placeholder="....."
          className={pwdErr ? 'border-red-400' : ''}
        />
        {pwdErr && <p className="text-xs text-red-500">{pwdErr}</p>}

        {/* Submit */}
        <button type="submit" disabled={loading}>
          {loading ? 'Signing in...' : 'Sign in →'}
        </button>
      </form>
    </div>
  </div>

```

```
);  
};
```

Key Features:

- 1. **Live validation:** Errors clear as user types
- 2. **Loading state:** Button disabled during request
- 3. **Error handling:** Shows toast on failure
- 4. **Auto-navigation:** Redirects to correct dashboard
- 5. **Responsive:** Mobile-friendly form

6.3 Registration Flow

```
javascript
```

```
// pages/auth/RegisterPage.jsx
```

```
const RegisterPage = () => {
  const [form, setForm] = useState({
    name: "",
    email: "",
    role: "",
    phone: "",
    password: "",
    confirmPassword: "",
  });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);

  const { register } = useAuth();
  const navigate = useNavigate();

  const validate = () => {
    const newErrors = {};

    if (!form.name.trim()) newErrors.name = 'Full name is required';
    if (!form.email.trim()) newErrors.email = 'Email is required';
    else if (!/^[^s@]+@[^s@]+\.[^s@]+$/.test(form.email)) {
      newErrors.email = 'Invalid email';
    }
    if (!form.role) newErrors.role = 'Select a role';
    if (!form.password) newErrors.password = 'Password is required';
    else if (form.password.length < 6) {
      newErrors.password = 'At least 6 characters';
    }
    if (form.password !== form.confirmPassword) {
      newErrors.confirmPassword = 'Passwords do not match';
    }

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!validate()) return;

    setLoading(true);
    const { confirmPassword, ...userData } = form;

    const result = await register(userData);
```

```
if (result.success) {  
  const redirectPath = result.user.role === 'supervisor'  
    ? '/supervisor'  
    : '/technician';  
  navigate(redirectPath);  
} else {  
  toast.error(result.error || 'Registration failed');  
}
```

```
setLoading(false);  
};
```

// Password strength indicator

```
const strength = getPasswordStrength(form.password);
```

```
return (  
  <form onSubmit={handleSubmit}>  
    {/* Name field */}  
    <input name="name" value={form.name} onChange={handleChange} />  
  
    {/* Email field */}  
    <input name="email" type="email" value={form.email} onChange={handleChange} />  
  
    {/* Role select */}  
    <select name="role" value={form.role} onChange={handleChange}>  
      <option value="">Select a role...</option>  
      <option value="technician">Technician</option>  
      <option value="supervisor">Supervisor</option>  
    </select>  
  
    {/* Password with strength indicator */}  
    <input name="password" type="password" value={form.password} onChange={handleChange} />  
    {form.password && (  
      <div className="flex gap-1 mb-1">  
        {[1, 2, 3, 4].map(i => (  
          <div key={i} className={`h-1 flex-1 rounded-full ${  
            i <= strength.score ? strength.color : 'bg-gray-200'  
          }`} />  
        ))}  
      </div>  
    )}  
  
    {/* Confirm password */}  
    <input name="confirmPassword" type="password" value={form.confirmPassword} />  
    {form.confirmPassword && (  
      <p className={form.password === form.confirmPassword
```

```

      ? 'text-green-600'
      : 'text-red-600'}>
    {form.password === form.confirmPassword ? '✓ Match' : '✗ No match'}
  </p>
)}

<button type="submit" disabled={loading}>
  {loading ? 'Creating account...' : 'Create account'}
</button>
</form>

);
};

```

Password Strength Algorithm:

javascript

```

const getPasswordStrength = (pwd) => {
  if (!pwd) return { score: 0, label: '', color: '' };
  let score = 0;
  if (pwd.length >= 8) score++;
  if (/[A-Z]/.test(pwd)) score++;
  if (/[0-9]/.test(pwd)) score++;
  if (/^[A-Za-z0-9]/.test(pwd)) score++;

  const labels = ['Too short', 'Weak', 'Fair', 'Good', 'Strong'];
  const colors = ['bg-red-400', 'bg-red-400', 'bg-yellow-400', 'bg-blue-400', 'bg-green-500'];

  return { score, label: labels[score], color: colors[score] };
};

```

6.4 Persistent Authentication

javascript

```

// App initialization
useEffect(() => {
  // AuthContext checks localStorage on mount
  const storedUser = localStorage.getItem('user');
  const token = localStorage.getItem('token');

  if (storedUser && token) {
    setUser(JSON.parse(storedUser));
  }
}, []);

```

Flow:

Page refresh



AuthContext initializes



Check localStorage



└─▶ Token found? ──▶ Parse user ──▶ Set state ──▶ User stays logged in



└─▶ No token? ──▶ Remain logged out

7. API Integration

7.1 API Service Layer

javascript

```
// services/api.js
```

```
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api/v1';
```

```
// Custom error class
```

```
class APIError extends Error {  
  constructor(message, status, data) {  
    super(message);  
    this.name = 'APIError';  
    this.status = status;  
    this.data = data;  
  }  
}
```

```
// Response handler
```

```
const handleResponse = async (response) => {  
  const contentType = response.headers.get('content-type');  
  const isJson = contentType && contentType.includes('application/json');  
  const data = isJson ? await response.json() : await response.text();
```

```
// Check if response failed OR server returned success: false
```

```
if (!response.ok || (data && data.success === false)) {  
  const errorMessage = data?.error || data?.message || data || `HTTP ${response.status}`;  
  throw new APIError(errorMessage, response.status, data);  
}
```

```
  return data;  
};
```

```
// Authenticated fetch wrapper
```

```
const fetchWithAuth = async (url, options = {}) => {  
  const token = localStorage.getItem('token');
```

```
  const headers = {  
    'Content-Type': 'application/json',  
    'Cache-Control': 'no-cache, no-store, must-revalidate',  
    ...options.headers,  
  };
```

```
  if (token) {  
    headers.Authorization = `Bearer ${token}`;  
  }
```

```
  try {  
    const response = await fetch(`${API_BASE_URL}${url}`, {  
      ...options,
```

```

    headers,
    credentials: 'omit',
  });

  return await handleResponse(response);
} catch (error) {
  // Handle 401 (token expired/invalid)
  if (error instanceof APIError && error.status === 401) {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    // Note: Don't redirect here (causes issues in AuthContext)
  }

  console.error('API Error:', error);
  throw error;
};

// Auth API
export const authAPI = {
  login: async (email, password) => {
    return await fetchWithAuth('/auth/login', {
      method: 'POST',
      body: JSON.stringify({ email, password }),
    });
  },

  register: async (userData) => {
    return await fetchWithAuth('/auth/register', {
      method: 'POST',
      body: JSON.stringify(userData),
    });
  }
};

// Job Card API
export const jobCardAPI = {
  getAll: async (filters = {}) => {
    const params = new URLSearchParams();
    Object.entries(filters).forEach(([key, value]) => {
      if (value !== undefined && value !== null) {
        params.append(key, value);
      }
    });

    const queryString = params.toString();
    const url = queryString ? `/job-cards?${queryString}` : '/job-cards';
    return await fetchWithAuth(url);
  }
};

```

```

    },

    getById: async (id) => {
      return await fetchWithAuth(`/job-cards/${id}`);
    },

    create: async (jobCardData) => {
      return await fetchWithAuth('/job-cards', {
        method: 'POST',
        body: JSON.stringify(jobCardData),
      });
    },

    update: async (id, updateData) => {
      return await fetchWithAuth(`/job-cards/${id}`, {
        method: 'PATCH',
        body: JSON.stringify(updateData),
      });
    },

    complete: async (id, completionData) => {
      return await fetchWithAuth(`/job-cards/${id}/complete`, {
        method: 'POST',
        body: JSON.stringify(completionData),
      });
    },

    delete: async (id) => {
      return await fetchWithAuth(`/job-cards/${id}`, {
        method: 'DELETE',
      });
    },

    getStatistics: async () => {
      return await fetchWithAuth('/job-cards/stats');
    }
  };

// Customer API
export const customerAPI = { /* ... */ };

// User API
export const userAPI = { /* ... */ };

```

Design Decisions:

1. **Centralized error handling:** All API errors go through `handleResponse`

2. **Auto-attach token:** `fetchWithAuth` adds Authorization header
3. **Custom error class:** Easier to handle specific error types
4. **Cache-Control headers:** Prevent stale data
5. **Environment variable:** API URL configurable (dev vs prod)

7.2 Error Handling Pattern

javascript

```
// Component using API
const JobCardList = () => {
  const [jobs, setJobs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const { toast } = useToast();

  useEffect(() => {
    const fetchJobs = async () => {
      setLoading(true);
      setError("");
      try {
        const res = await jobCardAPI.getAll();
        setJobs(res.data || []);
      } catch (err) {
        const errorMessage = typeof err === 'string' ? err : err.message || 'Failed to load jobs';
        setError(errorMessage);
        toast.error(errorMessage);
      } finally {
        setLoading(false);
      }
    };

    fetchJobs();
  }, []);

  if (loading) return <SkeletonJobList />;
  if (error) return <ErrorDisplay message={error} onRetry={fetchJobs} />;
  if (jobs.length === 0) return <EmptyState type="jobs" />;

  return <JobList jobs={jobs} />;
};
```

Error Handling Hierarchy:

Error occurs in API



Caught in component

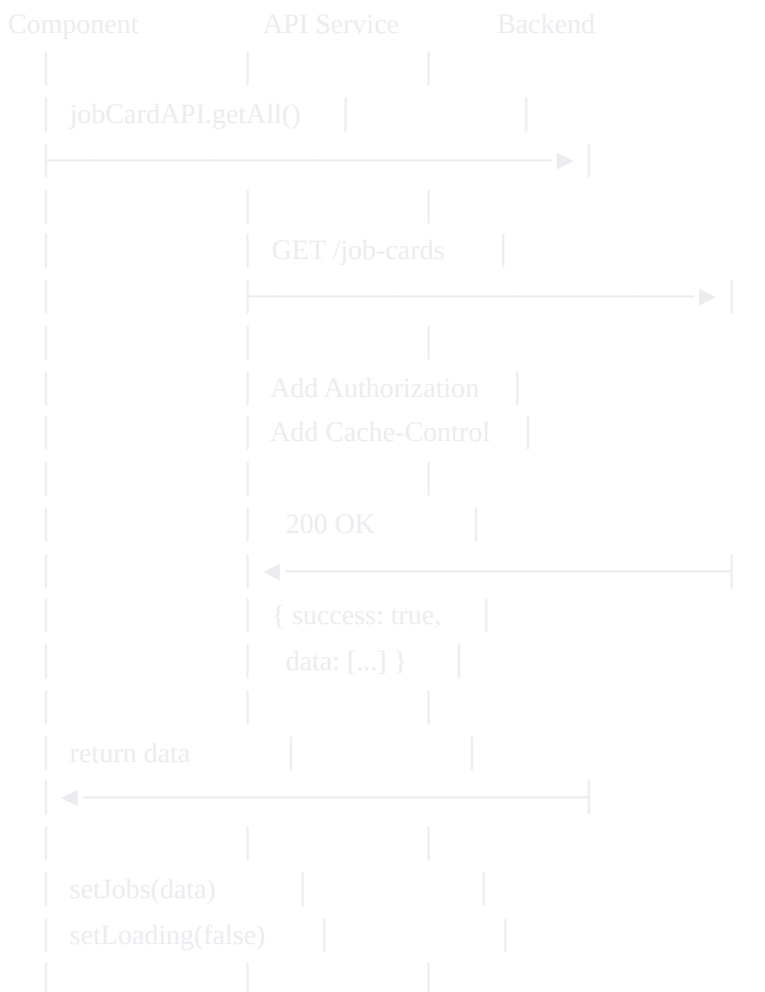


Set error state

Show toast notification

Render error UI (optional)

7.3 Request/Response Flow



8. UI/UX Design System

8.1 Design Philosophy

Brand Identity:

- Primary Color:** Amber (#F59E0B) - warm, energetic, Copy Cat brand
- Dark Background:** Slate-900 (#0F172A) - professional, modern

- **Typography:**
 - **Headings:** Syne (bold, geometric, attention-grabbing)
 - **Body:** DM Sans (clean, readable, friendly)

Design Principles:

1. **Minimalism:** Remove unnecessary elements
2. **Clarity:** One primary action per screen
3. **Feedback:** Every action has visual response (toast, animation)
4. **Consistency:** Same patterns across all pages
5. **Mobile-first:** Design for smallest screen, scale up

8.2 Color Palette

```
css

/* Primary */
--amber-400: #FBBF24; /* Lighter accent */
--amber-500: #F59E0B; /* Main brand color */
--amber-600: #D97706; /* Hover state */

/* Neutrals */
--slate-900: #0F172A; /* Dark backgrounds */
--slate-800: #1E293B; /* Sidebar hover */
--gray-50: #F9FADF; /* Page background */
--gray-100: #F3F4F6; /* Card borders */

/* Status colors */
--emerald-500: #10B981; /* Success / Completed */
--blue-500: #3B82F6; /* Info / In Progress */
--amber-500: #F59E0B; /* Warning / Pending */
--red-500: #EF4444; /* Error / Danger */
```

8.3 Typography System

```
javascript
```

```
// Import fonts in component
<style>{`
  @import url('https://fonts.googleapis.com/css2?family=Syne:wght@600;700;800&family=DM+Sans:wght@400;500;600&');

  .sup-root { font-family: 'DM Sans', system-ui, sans-serif; }
  .sup-title { font-family: 'Syne', system-ui, sans-serif; }
`}</style>

// Usage
<h1 className="sup-title text-2xl font-bold">Dashboard</h1>
<p className="text-sm text-gray-600">Overview of operations</p>
```

Font Weights:

- **DM Sans 400:** Body text
- **DM Sans 500:** Emphasis
- **DM Sans 600:** Bold text, buttons
- **Syne 600/700/800:** Headings, titles

8.4 Component Patterns

8.4.1 Button Styles

javascript

// Primary action

```
<button className="bg-amber-500 hover:bg-amber-600 text-white px-4 py-2.5
rounded-lg font-semibold text-sm transition-all shadow-sm
hover:shadow-amber-200 hover:shadow-md active:scale-[0.98]">
  Create Job
</button>
```

// Secondary action

```
<button className="border border-gray-200 text-gray-600 px-4 py-2.5
rounded-lg font-medium text-sm hover:bg-gray-50 transition-colors">
  Cancel
</button>
```

// Danger action

```
<button className="bg-red-500 hover:bg-red-600 text-white px-4 py-2.5
rounded-lg font-semibold text-sm transition-colors">
  Delete
</button>
```

// Loading state

```
<button disabled className="bg-amber-500 opacity-60 cursor-not-allowed ...">
  <svg className="animate-spin h-4 w-4" />
  Saving...
</button>
```

8.4.2 Status Badges

javascript

```

const STATUS_CONFIG = {
  pending: {
    pill: 'bg-amber-100 text-amber-700',
    dot: 'bg-amber-400',
    label: 'Pending'
  },
  in_progress: {
    pill: 'bg-blue-100 text-blue-700',
    dot: 'bg-blue-500',
    label: 'In Progress'
  },
  completed: {
    pill: 'bg-emerald-100 text-emerald-700',
    dot: 'bg-emerald-500',
    label: 'Completed'
  },
};

// Usage
const StatusBadge = ({ status }) => {
  const config = STATUS_CONFIG[status] || STATUS_CONFIG.pending;

  return (
    <span className={`inline-flex items-center gap-1.5 px-2.5 py-0.5
      rounded-full text-xs font-semibold ${config.pill}`}>
      <span className={`h-1.5 w-1.5 rounded-full ${config.dot}`} />
      {config.label}
    </span>
  );
};

```

8.4.3 Card Styles

javascript

```
// Standard card
<div className="bg-white rounded-xl border border-gray-100 shadow-sm p-5">
  {/* Content */}
</div>

// Interactive card (hover effect)
<button className="bg-white rounded-xl border border-gray-100 shadow-sm p-4
  w-full text-left hover:border-amber-200 hover:shadow-md transition-all">
  {/* Content */}
</button>

// Stat card
<div className="bg-white rounded-xl border border-gray-100 shadow-sm p-5
  hover:shadow-md transition-shadow">
  <div className="flex items-center justify-between mb-4">
    <span className="text-xs font-semibold text-gray-400 uppercase tracking-wider">
      Total Jobs
    </span>
    <div className="h-9 w-9 rounded-xl bg-slate-100 flex items-center justify-center">
      <svg className="h-4.5 w-4.5 text-slate-600" />
    </div>
  </div>
  <p className="sup-title text-3xl font-bold text-slate-900">42</p>
</div>
```

8.4.4 Form Inputs

```
javascript

// Standard input
<input className="block w-full rounded-lg border border-gray-200
  bg-slate-50 px-3.5 py-2.5 text-sm
  focus:outline-none focus:ring-2 focus:ring-amber-400 focus:bg-white
  transition-all" />

// Error state
<input className="block w-full rounded-lg border border-red-300
  bg-red-50 px-3.5 py-2.5 text-sm
  focus:outline-none focus:ring-2 focus:ring-red-400" />

// With icon
<div className="relative">
  <svg className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-300" />
  <input className="block w-full pl-9 pr-4 py-2.5 rounded-lg ..." />
</div>
```

8.5 Animation Strategy

```
javascript

// Page load animations
<div className="fade-up" style={{ animationDelay: '0.05s' }}>
  {/* Content */}
</div>

<style>{`
  @keyframes fadeUp {
    from { opacity: 0; transform: translateY(16px); }
    to   { opacity: 1; transform: translateY(0); }
  }
  .fade-up { animation: fadeUp 0.5s cubic-bezier(0,0,0.2,1) both; }
`}</style>

// Hover transitions
<button className="transition-all duration-150 hover:scale-105">

// Modal entrance
<div style={{ animation: 'modalIn 0.2s cubic-bezier(0,0,0.2,1)' }}>
  <style>{`
    @keyframes modalIn {
      from { opacity: 0; transform: scale(0.96) translateY(8px); }
      to   { opacity: 1; transform: scale(1) translateY(0); }
    }
  `}</style>
</div>
```

Animation Guidelines:

- **Duration:** 150-350ms (longer = sluggish)
- **Easing:** `cubic-bezier(0,0,0.2,1)` for most (gentle acceleration)
- **When to animate:** State changes, page transitions, modals
- **When NOT to:** Continuous loops, excessive bouncing

8.6 Responsive Design

```
javascript
```

```
// Tailwind breakpoints
sm: 640px // Small tablets
md: 768px // Tablets
lg: 1024px // Desktop
xl: 1280px // Large desktop

// Usage
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {/* Responsive grid: 1 col mobile, 2 col tablet, 3 col desktop */}
</div>

<div className="hidden lg:flex">
  {/* Only show on desktop */}
</div>

<div className="lg:hidden">
  {/* Only show on mobile/tablet */}
</div>
```

Mobile-First Approach:

1. Design for mobile (320px+)
2. Add tablet styles ((md:))
3. Add desktop styles ((lg:), (xl:))

9. Key Features Implementation

9.1 Supervisor Dashboard

```
javascript
```

```
// pages/supervisor/SupervisorDashboard.jsx
```

```
const SupervisorDashboard = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const isSubPage =
    location.pathname.includes('/jobs/new') ||
    location.pathname.match(/\/jobs\/\d+/) ||
    // ... other sub-pages

  return (
    <div className="min-h-screen flex bg-gray-50">
      {/* Sidebar (always visible) */}
      <aside className="w-52 bg-slate-900 flex flex-col sticky top-0 h-screen">
        {/* Logo */}
        <div className="px-5 py-5 border-b border-slate-800">
          <Logo size="sm" />
          <p className="text-white font-bold text-xs">COPY CAT</p>
        </div>

        {/* Navigation */}
        <nav className="flex-1 px-3 py-4 space-y-0.5">
          {NAV_ITEMS.map(item => (
            <NavLink key={item.to} to={item.to} className={({ isActive }) =>
              `flex items-center gap-2.5 px-3 py-2.5 rounded-lg text-sm transition-all
              ${isActive
                ? 'bg-amber-500 text-white font-semibold'
                : 'text-slate-400 hover:text-white hover:bg-slate-800'
              }`
            >
              <svg className="h-4 w-4">{item.icon}</svg>
              {item.label}
            </NavLink>
          ))}
        </nav>

        {/* User + logout */}
        <div className="px-3 pb-4 border-t border-slate-800 pt-3">
          <div className="flex items-center gap-2.5 px-2 mb-3">
            <div className="h-7 w-7 rounded-full bg-amber-500/20">
              <span className="text-xs font-bold text-amber-400">
                {user?.name?.charAt(0).toUpperCase()}
              </span>
            </div>

```

```

    <div>
      <p className="text-xs font-semibold text-white">{user?.name}</p>
      <p className="text-xs text-slate-500">{user?.role}</p>
    </div>
  </div>
  <button onClick={() => { logout(); navigate('/login'); }}>
    Sign out
  </button>
</div>
</aside>

{/* Main content */}
<main className="flex-1 p-6 overflow-auto">
  {/* Back button for sub-pages */}
  {isSubPage && (
    <button onClick={() => navigate(-1)}>
      ← Back
    </button>
  )}

  <Routes>
    <Route index element={<StatsOverview />} />
    <Route path="jobs" element={<AllJobsList />} />
    <Route path="jobs/new" element={<CreateJobCard />} />
    <Route path="jobs/:id" element={<SupervisorJobDetail />} />
    <Route path="customers" element={<CustomerList />} />
    <Route path="users" element={<UserList />} />
    <Route path="users/:id" element={<UserDetail />} />
  </Routes>
</main>
</div>
);
};

```

Key Features:

1. **Sticky sidebar:** Navigation always visible (desktop)
2. **Active state:** Current route highlighted
3. **User info:** Avatar with initials
4. **Contextual back button:** Shows on detail pages
5. **Nested routing:** Sidebar stays mounted

9.2 Statistics Overview

javascript

```
// pages/supervisor/StatsOverview.jsx
```

```
const StatsOverview = () => {
  const [jobStats, setJobStats] = useState(null);
  const [userStats, setUserStats] = useState(null);
  const [custStats, setCustStats] = useState(null);
  const [recent, setRecent] = useState([]);
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  useEffect(() => {
    (async () => {
      try {
        const [js, us, cs, rj] = await Promise.all([
          jobCardAPI.getStatistics(),
          userAPI.getStatistics(),
          customerAPI.getStatistics(),
          jobCardAPI.getAll({ limit: 6 }),
        ]);
        setJobStats(js.data?.stats);
        setUserStats(us.data?.stats);
        setCustStats(cs.data?.stats);
        setRecent(rj.data || []);
      } catch (e) {
        console.error(e);
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  const navigateToJobs = (statusFilter) => {
    navigate('/supervisor/jobs', { state: { statusFilter } });
  };

  return (
    <div>
      { /* Header */ }
      <div className="flex items-center justify-between mb-6">
        <h1 className="sup-title text-2xl font-bold text-slate-900">Dashboard</h1>
        <button onClick={() => navigate('/supervisor/jobs/new')}>
          New Job
        </button>
      </div>

      { /* Stat cards */ }
```

```

{loading ? <SkeletonStatCards count={6} /> : (
  <div className="grid grid-cols-2 lg:grid-cols-3 gap-4 mb-7">
    <StatCard
      label="Total Jobs"
      value={jobStats?.total_jobs}
      icon="..."
      onClick={() => navigateToJobs('all')}
    />
    <StatCard
      label="Pending"
      value={jobStats?.pending_jobs}
      icon="..."
      onClick={() => navigateToJobs('pending')}
    />
    {/* More stat cards... */}
  </div>
)}

{/* Recent jobs list */}
<div className="bg-white rounded-xl border border-gray-100 shadow-sm">
  <div className="flex items-center justify-between px-5 py-3.5 border-b border-gray-50">
    <h2 className="text-sm font-bold text-slate-800">Recent Jobs</h2>
    <button onClick={() => navigate('/supervisor/jobs')}>
      View all →
    </button>
  </div>
  {recent.map(job => (
    <button key={job.id} onClick={() => navigate(`/supervisor/jobs/${job.id}`)}
      className="w-full flex items-center justify-between px-5 py-3.5
        hover:bg-amber-50/50 transition-colors">
      <div>
        <p className="text-sm font-medium text-slate-800">{job.title}</p>
        <p className="text-xs text-gray-400">
          {job.customer?.name} · {job.technician?.name}
        </p>
      </div>
      <StatusBadge status={job.status} />
    </button>
  ))}
</div>
</div>
);
};

```

Features:

1. **Parallel fetching:** `Promise.all()` loads all stats at once

2. **Clickable stats:** Navigate to filtered job list
3. **Recent activity:** Quick access to latest jobs
4. **Loading skeleton:** Shows structure while loading
5. **Navigation state:** Pass filter to job list

9.3 Job Card List with Filtering

javascript

```
// pages/supervisor/AllJobsList.jsx
```

```
const AllJobsList = () => {
  const location = useLocation();
  const navigate = useNavigate();

  const initialStatus = location.state?.statusFilter || 'all';

  const [jobs, setJobs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState("");
  const [status, setStatus] = useState(initialStatus);
  const [page, setPage] = useState(1);
  const [pagination, setPag] = useState(null);

  const fetch = useCallback(async () => {
    setLoading(true);
    try {
      const f = { page, limit: 20 };
      if (status !== 'all') f.status = status;
      if (search.trim()) f.search = search.trim();
      const res = await jobCardAPI.getAll(f);
      setJobs(res.data || []);
      setPag(res.pagination);
    } catch (e) {
      console.error(e);
    } finally {
      setLoading(false);
    }
  }, [page, status, search]);

  useEffect(() => { fetch(); }, [fetch]);
  useEffect(() => { setPage(1); }, [search, status]); // Reset page when filters change

  const TABS = ['all', 'pending', 'in_progress', 'completed'];

  return (
    <div>
      { /* Header */ }
      <div className="flex items-center justify-between mb-6">
        <h1 className="sup-title text-2xl font-bold">Job Cards</h1>
        <button onClick={() => navigate('/supervisor/jobs/new')}>
          New Job
        </button>
      </div>
    </div>
  )
}
```

```

{/* Search + Filters */}
<div className="bg-white rounded-xl border border-gray-100 shadow-sm p-4 mb-4 space-y-3">
  {/* Search input */}
  <div className="relative">
    <svg className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-300" />
    <input
      type="text"
      value={search}
      onChange={e => setSearch(e.target.value)}
      placeholder="Search by title or description..."
      className="block w-full pl-9 pr-4 py-2.5 rounded-lg border ..."
    />
  </div>

  {/* Status tabs */}
  <div className="flex gap-1">
    {TABS.map(t => (
      <button key={t} onClick={() => setStatus(t)}
        className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all
          ${status === t
            ? 'bg-slate-900 text-white'
            : 'bg-gray-100 text-gray-500 hover:bg-gray-200'}
          `}>
        {t === 'all' ? 'All' : t === 'in_progress' ? 'In Progress' : t.charAt(0).toUpperCase()+t.slice(1)}
      </button>
    ))}
  </div>
</div>

{/* Jobs table */}
{loading ? <SkeletonTable rows={6} cols={6} /> : jobs.length === 0 ? (
  <EmptyState
    type={search ? 'search' : 'jobs'}
    query={search}
    onAction={!search ? () => navigate('/supervisor/jobs/new') : undefined}
    actionLabel="Create Job Card"
  />
) : (
  <div className="bg-white rounded-xl border border-gray-100 shadow-sm overflow-hidden">
    <table className="w-full text-sm">
      <thead>
        <tr className="border-b border-gray-100">
          {[ 'Job', 'Customer', 'Technician', 'Status', 'Priority', 'Scheduled' ].map(h => (
            <th key={h} className="px-4 py-3 text-left text-xs font-bold text-gray-400 uppercase bg-gray-50">
              {h}
            </th>
          ))}
        </tr>
      </thead>
    </table>
  </div>
)

```

```

    </tr>
  </thead>
  <tbody className="divide-y divide-gray-50">
    {jobs.map(job => (
      <tr key={job.id}
        onClick={() => navigate(`/supervisor/jobs/${job.id}`)}
        className="hover:bg-amber-50/40 cursor-pointer transition-colors">
          <td className="px-4 py-3.5">
            <p className="font-semibold text-slate-800">{job.title}</p>
            {job.description && (
              <p className="text-xs text-gray-400 truncate max-w-[200px]">
                {job.description}
              </p>
            )}
          </td>
          <td className="px-4 py-3.5 text-gray-600">{job.customer?.name}</td>
          <td className="px-4 py-3.5 text-gray-600">{job.technician?.name}</td>
          <td className="px-4 py-3.5"><StatusBadge status={job.status} /></td>
          <td className="px-4 py-3.5">
            <span className={PRIORITY_COLOR[job.priority]}>
              {job.priority}
            </span>
          </td>
          <td className="px-4 py-3.5 text-gray-400 text-xs">
            {formatDate(job.scheduled_date)}
          </td>
        </tr>
      )))
  </tbody>
</table>

{/* Pagination */}
{pagination && pagination.totalPages > 1 && (
  <div className="flex items-center justify-between px-4 py-3 border-t border-gray-50">
    <p className="text-xs text-gray-400">
      Page {pagination.currentPage} of {pagination.totalPages}
    </p>
    <div className="flex gap-1.5">
      <button disabled={page <= 1} onClick={() => setPage(p => p - 1)}>
        ← Prev
      </button>
      <button disabled={page >= pagination.totalPages} onClick={() => setPage(p => p + 1)}>
        Next →
      </button>
    </div>
  </div>
)}

```

```
    </div>
  })
</div>
);
};
```

Features:

1. **Receive filter from navigation:** `location.state.statusFilter`
2. **Debounced search:** User types, page resets, fetch triggered
3. **Tab filters:** Quick status filtering
4. **Pagination:** Load 20 at a time
5. **Empty states:** Different for no results vs no jobs
6. **Clickable rows:** Navigate to job detail

9.4 Technician Dashboard (Mobile-Optimized)

javascript

```
// pages/technician/TechnicianDashboard.jsx
```

```
const TechnicianDashboard = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const isDetail = /\stechnician\/.+/.test(location.pathname);

  return (
    <div className="tech-root min-h-screen bg-gray-50">
      { /* Top nav (mobile-friendly) */ }
      <header className="sticky top-0 z-20 bg-slate-900 border-b border-slate-800">
        <div className="max-w-2xl mx-auto px-4 h-14 flex items-center justify-between">
          { /* Left */ }
          { isDetail ? (
            <button onClick={() => navigate(-1)} className="flex items-center gap-1.5">
              <svg className="h-4 w-4" />
              <span className="text-sm">Back</span>
            </button>
          ) : (
            <div className="flex items-center gap-2.5">
              <Logo size="sm" />
              <div>
                <p className="text-white text-xs font-bold">MY JOBS</p>
                <p className="text-slate-500 text-[10px]">Copy Cat Group</p>
              </div>
            </div>
          ) }
        </div>
        { /* Right */ }
        <div className="flex items-center gap-3">
          <span className="text-xs text-slate-400">{user?.name}</span>
          <button onClick={() => { logout(); navigate('/login'); }}>
            Sign out
          </button>
        </div>
      </div>
    </header>

    { /* Main (max-width for mobile) */ }
    <main className="max-w-2xl mx-auto px-4 py-5">
      <Routes>
        <Route index element={ <JobCardList /> } />
        <Route path=":id" element={ <JobDetail /> } />
      </Routes>
    </main>
  )
}
```

```
</div>  
);  
};
```

Mobile Optimizations:

- 1. **max-width 2xl:** Prevents content stretching on desktop
- 2. **Sticky header:** Navigation always accessible
- 3. **Contextual back button:** Shows on detail view
- 4. **Simplified nav:** No sidebar (mobile doesn't have space)
- 5. **Touch-friendly:** Larger tap targets (py-2.5 = 40px)

9.5 Job Completion Flow (Technician)

```
javascript
```

```
// pages/technician/JobDetail.jsx
```

```
const JobDetail = () => {  
  const { id } = useParams();  
  const { toast } = useToast();  
  const navigate = useNavigate();  
  
  const [job, setJob] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [starting, setStarting] = useState(false);  
  const [showForm, setShowForm] = useState(false);  
  const [completing, setCompleting] = useState(false);  
  const [form, setForm] = useState({ work_performed: "", notes: "" });  
  const [formErr, setFormErr] = useState({});
```

```
// Load job
```

```
useEffect(() => {  
  (async () => {  
    try {  
      const res = await jobCardAPI.getById(id);  
      setJob(res.data.jobCard);  
    } catch (e) {  
      toast.error('Failed to load job');  
    } finally {  
      setLoading(false);  
    }  
  })();  
}, [id]);
```

```
// Start job
```

```
const startJob = async () => {  
  setStarting(true);  
  try {  
    const res = await jobCardAPI.update(id, {  
      status: 'in_progress',  
      actual_start_time: new Date().toISOString(),  
    });  
    setJob(res.data.jobCard);  
    toast.success('Job started — start time recorded');  
  } catch (e) {  
    toast.error('Failed to start job');  
  } finally {  
    setStarting(false);  
  }  
};
```

```

// Validate completion form
const validate = () => {
  const e = {};
  if (!form.work_performed.trim()) {
    e.work_performed = 'Required';
  } else if (form.work_performed.trim().length < 10) {
    e.work_performed = 'Minimum 10 characters';
  }
  setFormErr(e);
  return !Object.keys(e).length;
};

// Complete job
const completeJob = async (e) => {
  e.preventDefault();
  if (!validate()) return;

  setCompleting(true);
  try {
    const res = await jobCardAPI.complete(id, {
      actual_end_time: new Date().toISOString(),
      work_performed: form.work_performed.trim(),
      notes: form.notes.trim() || undefined,
    });
    setJob(res.data.jobCard);
    setShowForm(false);
    toast.success('Job completed successfully!');
  } catch (e) {
    toast.error('Failed to complete job');
  } finally {
    setCompleting(false);
  }
};

if (loading) return <SkeletonDetail />;
if (!job) return <div>Job not found</div>;

const isPending = job.status === 'pending';
const isInProgress = job.status === 'in_progress';
const isCompleted = job.status === 'completed';

return (
  <div className="space-y-3">
    {/* Status bar */}
    <div className={`h-1 rounded-full ${STATUS[job.status].bar}`} />

    {/* Job info card */}

```

```

<div className="bg-white rounded-xl border border-gray-100 shadow-sm p-4">
  <h1 className="text-lg font-bold text-slate-900">{job.title}</h1>
  {job.description && <p className="text-sm text-gray-500">{job.description}</p>}
  {/* ...more details */}
</div>

{/* Customer card */}
<div className="bg-white rounded-xl border border-gray-100 shadow-sm p-4">
  <p className="text-xs font-bold text-gray-400 uppercase mb-3">Customer</p>
  <p>{job.customer.name}</p>
  <p>{job.customer.phone}</p>
  <p>{job.customer.address}</p>
</div>

{/* START JOB (pending only) */}
{isPending && (
  <div className="bg-white rounded-xl border border-gray-100 shadow-sm p-4">
    <p className="text-xs font-bold text-gray-400 uppercase mb-1">Ready to begin?</p>
    <p className="text-xs text-gray-400 mb-4">
      This records your start time and marks the job active.
    </p>
    <button onClick={startJob} disabled={starting} className="w-full ...">
      {starting ? 'Starting...' : 'Start Job'}
    </button>
  </div>
)}

{/* COMPLETE JOB (in_progress only) */}
{isInProgress && (
  <div className="bg-white rounded-xl border border-gray-100 shadow-sm p-4">
    <p className="text-xs font-bold text-gray-400 uppercase mb-1">Finish up</p>
    {!showForm ? (
      <>
        <p className="text-xs text-gray-400 mb-4">
          Fill in the completion report to close this job.
        </p>
        <button onClick={() => setShowForm(true)} className="w-full ...">
          Complete this job
        </button>
      </>
    ) : (
      <form onSubmit={completeJob} className="space-y-4 mt-3">
        {/* Work performed */}
        <div>
          <label className="text-xs font-semibold text-gray-600 uppercase">
            Work performed <span className="text-red-400">*</span>
          </label>

```

```

<textarea
  rows={4}
  value={form.work_performed}
  onChange={e => {
    setForm(p => ({ ...p, work_performed: e.target.value }));
    setFormErr(p => ({ ...p, work_performed: " " }));
  }}
  placeholder="Describe exactly what was done..."
  className={formErr.work_performed ? 'border-red-300' : ''}
/>

{formErr.work_performed && (
  <p className="text-xs text-red-500">{formErr.work_performed}</p>
)}
</div>

{/* Notes (optional) */}
<div>
  <label className="text-xs font-semibold text-gray-600 uppercase">
    Notes <span className="text-gray-300">(optional)</span>
  </label>
  <textarea
    rows={2}
    value={form.notes}
    onChange={e => setForm(p => ({ ...p, notes: e.target.value })))}
    placeholder="Follow-up actions, observations..."
  />
</div>

{/* Actions */}
<div className="flex gap-2.5 pt-1">
  <button type="button" onClick={() => setShowForm(false)}>
    Cancel
  </button>
  <button type="submit" disabled={completing}>
    {completing ? 'Saving...' : 'Submit & close'}
  </button>
</div>
</form>
)}
</div>
)}

{/* COMPLETED summary */}
{isCompleted && (
  <div className="rounded-xl overflow-hidden border border-emerald-100">
    <div className="bg-emerald-600 px-4 py-3 flex items-center gap-2">
      <svg className="h-4 w-4 text-white" />

```

```

    <span className="text-white text-sm font-semibold">Job Complete</span>
  </div>
  <div className="bg-emerald-50 p-4 space-y-3">
    <div>
      <p className="text-xs font-bold text-emerald-700 uppercase">Work performed</p>
      <p className="text-sm text-emerald-900">{job.work_performed}</p>
    </div>
    {job.notes && (
      <div>
        <p className="text-xs font-bold text-emerald-700 uppercase">Notes</p>
        <p className="text-sm text-emerald-900">{job.notes}</p>
      </div>
    )}
  </div>
</div>
)}
);
};

```

Flow:

```

Pending
|
| [Start Job] button
▼
In Progress
|
| [Complete this job] button
▼
Show completion form
|
| Fill work_performed + notes
| [Submit & close]
▼
Completed
|
└─► Show summary (read-only)

```

UX Decisions:

1. **Status bar:** Visual indicator at top
2. **Progressive disclosure:** Completion form hidden until needed
3. **Required fields:** `work_performed` enforced (min 10 chars)
4. **Auto-timestamps:** Start/end times set automatically

5. **Immutable:** Completed jobs show summary (no edit)
-

10. Performance Optimizations

10.1 Code Splitting

```
javascript

// Lazy load routes (future enhancement)
import { lazy, Suspense } from 'react';

const SupervisorDashboard = lazy(() => import('./pages/supervisor/SupervisorDashboard'));
const TechnicianDashboard = lazy(() => import('./pages/technician/TechnicianDashboard'));

<Suspense fallback={<LoadingScreen />}>
  <Routes>
    <Route path="/supervisor/*" element={<SupervisorDashboard />} />
    <Route path="/technician/*" element={<TechnicianDashboard />} />
  </Routes>
</Suspense>
```

Why Lazy Load?

- **Smaller initial bundle:** Faster first paint
- **On-demand loading:** Only load supervisor code if user is supervisor
- **Trade-off:** Slight delay when navigating to lazy routes

10.2 Memoization

```
javascript

// Expensive computation (filter/sort)
const filteredJobs = useMemo(() => {
  return jobs
    .filter(job => status === 'all' || job.status === status)
    .filter(job => !search || job.title.toLowerCase().includes(search.toLowerCase()))
    .sort((a, b) => new Date(b.created_at) - new Date(a.created_at));
}, [jobs, status, search]);

// Callback memoization
const handleDelete = useCallback((id) => {
  deleteJob(id);
}, []);
```

When to Memoize:

- **useMemo:** Expensive calculations (filtering/sorting large lists)
- **useCallback:** Callbacks passed to child components (prevent re-renders)
- **When NOT:** Simple renders, small lists (overhead > benefit)

10.3 Debouncing

```
javascript

// Search input debouncing (future enhancement)
import { useEffect, useState } from 'react';

const useDebounce = (value, delay) => {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(handler);
  }, [value, delay]);

  return debouncedValue;
};

// Usage
const [search, setSearch] = useState("");
const debouncedSearch = useDebounce(search, 500);

useEffect(() => {
  fetchJobs({ search: debouncedSearch });
}, [debouncedSearch]);
```

Why Debounce?

- **Reduce API calls:** Don't fetch on every keystroke
- **Better UX:** No flickering/loading states
- **Server-friendly:** Fewer requests

10.4 Image Optimization

```
javascript
```

```
// Logo component
<img
  src={logoImage}
  alt="Copy Cat Group"
  className="h-10 w-auto"
  loading="lazy" // Native lazy loading
  decoding="async" // Async decode
/>

// Future: Use WebP with PNG fallback
<picture>
  <source srcSet="logo.webp" type="image/webp" />
  
</picture>
```

10.5 Bundle Size Optimization

Current Build:

```
bash

npm run build

# Output
dist/assets/index-abc123.js 142 KB (gzipped: 45 KB)
dist/assets/index-xyz789.css 18 KB (gzipped: 4 KB)
```

Optimization Strategies:

1. **Tree shaking:** Vite automatically removes unused code
2. **Minification:** Production builds minified
3. **Code splitting:** Lazy load routes (future)
4. **Dependency audit:** Avoid large libraries

Example:

```
javascript

// ❌ Bad (imports entire library)
import _ from 'lodash';
_.debounce(fn, 500);

// ✅ Good (imports only needed function)
import debounce from 'lodash/debounce';
debounce(fn, 500);
```

11. Code Walkthrough

11.1 Component Lifecycle Example

Let's trace **creating a job card** from form submission to API response:

1. User Fills Form

```
javascript

// pages/supervisor/CreateJobCard.jsx

const [form, setForm] = useState({
  customer_id: "",
  technician_id: "",
  title: "",
  description: "",
  priority: 'medium',
  scheduled_date: "",
  estimated_duration: "",
  notes: "",
});

// User types in title field
<input
  name="title"
  value={form.title}
  onChange={e => setForm(p => ({ ...p, title: e.target.value })))}
/>
```

2. Form Submission

```
javascript
```

```
const handleSubmit = async (e) => {
  e.preventDefault();

  // Client-side validation
  if (!validate()) return;

  setSubmitting(true);

  try {
    const payload = {
      customer_id: Number(form.customer_id),
      technician_id: Number(form.technician_id),
      title: form.title.trim(),
      priority: form.priority,
      scheduled_date: form.scheduled_date,
    };

    if (form.description.trim()) payload.description = form.description.trim();
    if (form.estimated_duration) payload.estimated_duration = Number(form.estimated_duration);
    if (form.notes.trim()) payload.notes = form.notes.trim();

    // Call API
    const res = await jobCardAPI.create(payload);

    // Navigate to new job
    navigate(`/supervisor/jobs/${res.data.jobCard.id}`);

  } catch (err) {
    toast.error('Failed to create job card');
  } finally {
    setSubmitting(false);
  }
};
```

3. API Call

javascript

```
// services/api.js

export const jobCardAPI = {
  create: async (jobCardData) => {
    return await fetchWithAuth('/job-cards', {
      method: 'POST',
      body: JSON.stringify(jobCardData),
    });
  },
};

// fetchWithAuth adds Authorization header
const token = localStorage.getItem('token');
headers.Authorization = `Bearer ${token}`;

// Send POST request
fetch('http://localhost:5000/api/v1/job-cards', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer eyJhbGciOiJIUzI1...',
  },
  body: JSON.stringify({
    customer_id: 7,
    technician_id: 5,
    title: 'Photocopier Maintenance',
    priority: 'medium',
    scheduled_date: '2026-02-20T08:00:00Z',
  }),
});
```

4. Backend Processing

Backend validates → Creates job → Returns with ID

5. Response Handling

javascript

```
// API returns
{
  success: true,
  message: 'Job card created successfully',
  data: {
    jobCard: {
      id: 42,
      title: 'Photocopier Maintenance',
      status: 'pending',
      customer: { ... },
      technician: { ... },
      ...
    }
  }
}

// Navigate to new job
navigate(`/supervisor/jobs/42`);
```

6. Job Detail Page Loads

```
javascript

// pages/supervisor/SupervisorJobDetail.jsx

useEffect(() => {
  (async () => {
    const res = await jobCardAPI.getById(42);
    setJob(res.data.jobCard);
  })();
}, [id]);

// Renders job details
<h1>{job.title}</h1>
<StatusBadge status={job.status} />
<CustomerInfo customer={job.customer} />
```

11.2 State Update Flow

Scenario: Technician completes a job



11.3 Context Usage Example

javascript

```
// App.jsx wraps everything in AuthProvider
```

```
<AuthProvider>  
  <BrowserRouter>  
    <Routes>...</Routes>  
  </BrowserRouter>  
</AuthProvider>
```

```
// Login page updates context
```

```
const { login } = useAuth();  
const result = await login(email, password);  
// → AuthContext sets user state  
// → localStorage updated  
// → Navigate to dashboard
```

```
// Dashboard reads context
```

```
const { user, isSupervisor } = useAuth();
```

```
if (isSupervisor) {  
  return <SupervisorDashboard />;  
} else {  
  return <TechnicianDashboard />;  
}
```

```
// Logout clears context
```

```
const { logout } = useAuth();  
logout();  
// → localStorage cleared  
// → user state set to null  
// → ProtectedRoute redirects to login
```

12. Build & Deployment

12.1 Development Setup

```
bash
```

```
# Install dependencies
```

```
npm install
```

```
# Start dev server
```

```
npm run dev
```

```
# → Runs on http://localhost:5173
```

```
# Build for production
```

```
npm run build
```

```
# → Generates dist/ folder
```

```
# Preview production build
```

```
npm run preview
```

12.2 Environment Variables

```
bash
```

```
# .env.development
```

```
VITE_API_URL=http://localhost:5000/api/v1
```

```
# .env.production
```

```
VITE_API_URL=https://api.copycatgroup.com/api/v1
```

Access in code:

```
javascript
```

```
const API_BASE_URL = import.meta.env.VITE_API_URL;
```

12.3 Build Configuration

```
javascript
```

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: false, // Disable in production (smaller bundle)
    minify: 'esbuild', // Fast minification
  },
  server: {
    port: 5173,
    open: true, // Auto-open browser
  },
});
```

12.4 Deployment Options

Option 1: Netlify (Recommended)

```
bash

# Install Netlify CLI
npm install -g netlify-cli

# Deploy
npm run build
netlify deploy --prod --dir=dist
```

netlify.toml:

```
toml

[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Why redirect rule?

- **SPA routing:** All routes handled by React Router

- Without redirect: `/supervisor/jobs` returns 404 on page refresh

Option 2: Vercel

```
bash

# Install Vercel CLI
npm install -g vercel

# Deploy
vercel --prod
```

vercel.json:

```
json

{
  "rewrites": [
    { "source": "/(.*)", "destination": "/" }
  ]
}
```

Option 3: DigitalOcean App Platform

1. Connect GitHub repo
2. Configure build: `npm run build`
3. Configure output: `dist/`
4. Add environment variables
5. Deploy

12.5 Production Checklist

Pre-deployment:

- ☐ Environment variables configured
- ☐ API_URL points to production backend
- ☐ Remove console.logs (or use environment check)
- ☐ Test all user flows
- ☐ Check responsive design (mobile, tablet, desktop)
- ☐ Verify authentication redirects
- ☐ Test error handling (network errors, API errors)
- ☐ Check accessibility (keyboard navigation, screen readers)

Performance:

- ☐ Bundle size < 200 KB (gzipped)

- ☐ Images optimized
- ☐ Fonts loaded efficiently
- ☐ No memory leaks (useEffect cleanup)

SEO (if applicable):

- ☐ Meta tags (title, description)
 - ☐ Open Graph tags (social media previews)
 - ☐ Favicon
-

13. Future Enhancements

13.1 Short-Term (Next Sprint)

1. Offline Support (Service Worker)

```
javascript

// public/sw.js
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open('jobcard-v1').then((cache) => {
      return cache.addAll([
        '/',
        '/index.html',
        '/assets/index.js',
        '/assets/index.css',
      ]);
    })
  );
});

self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then((response) => {
      return response || fetch(event.request);
    })
  );
});
```

Benefits:

- Work offline (view cached jobs)
- Faster load times
- Progressive Web App (PWA) installable

2. Real-Time Updates (WebSockets)

```
javascript

import io from 'socket.io-client';

const socket = io(import.meta.env.VITE_WS_URL);

socket.on('job-status-changed', ({ jobCardId, newStatus }) => {
  // Update local state
  setJobs(prev => prev.map(job =>
    job.id === jobCardId ? { ...job, status: newStatus } : job
  ));

  // Show notification
  toast.info(`Job #${jobCardId} status changed to ${newStatus}`);
});
```

3. Camera Integration (Before/After Photos)

```
javascript

const [photo, setPhoto] = useState(null);

const capturePhoto = async () => {
  const stream = await navigator.mediaDevices.getUserMedia({ video: true });
  // Capture image from stream
  // Convert to base64
  // Upload to backend
};

<button onClick={capturePhoto}><img alt="camera icon" data-bbox="311 644 331 656"/> Take Photo</button>
```

13.2 Medium-Term (Future Versions)

1. Dark Mode

```
javascript

const [theme, setTheme] = useState('light');

useEffect(() => {
  document.documentElement.classList.toggle('dark', theme === 'dark');
}, [theme]);

// Tailwind dark mode classes
<div className="bg-white dark:bg-slate-900 text-gray-900 dark:text-white">
```

2. Advanced Filtering

javascript

```
// Multi-select filters
<select multiple value={selectedTechnicians} onChange={handleTechChange}>
  {technicians.map(tech => (
    <option key={tech.id} value={tech.id}>{tech.name}</option>
  ))}
</select>

// Date range picker
<DateRangePicker
  startDate={startDate}
  endDate={endDate}
  onChange={({ startDate, endDate }) => {
    setStartDate(startDate);
    setEndDate(endDate);
  }}
/>
```

3. Export to PDF/Excel

javascript

```
import jsPDF from 'jspdf';
import * as XLSX from 'xlsx';

const exportToPDF = () => {
  const doc = new jsPDF();
  doc.text('Job Cards Report', 10, 10);
  jobs.forEach((job, i) => {
    doc.text(`${job.id} - ${job.title}`, 10, 20 + (i * 10));
  });
  doc.save('job-cards.pdf');
};

const exportToExcel = () => {
  const ws = XLSX.utils.json_to_sheet(jobs);
  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, ws, 'Jobs');
  XLSX.writeFile(wb, 'job-cards.xlsx');
};
```

4. Push Notifications (Mobile)

javascript

```

// Request permission
const requestNotificationPermission = async () => {
  const permission = await Notification.requestPermission();
  if (permission === 'granted') {
    // Register service worker
    // Subscribe to push notifications
  }
};

// Receive notification
navigator.serviceWorker.addEventListener('message', (event) => {
  new Notification('New Job Assigned', {
    body: event.data.jobTitle,
    icon: '/logo.png',
  });
});

```

13.3 Long-Term (Roadmap)

1. Native Mobile Apps (React Native)

```

javascript

// Shared business logic
import { jobCardAPI } from '../services/api';

// Platform-specific UI
import { View, Text, TouchableOpacity } from 'react-native';

const JobCard = ({ job }) => (
  <TouchableOpacity onPress={() => navigate(`/jobs/${job.id}`)}>
    <View>
      <Text>{job.title}</Text>
      <StatusBadge status={job.status} />
    </View>
  </TouchableOpacity>
);

```

2. Advanced Analytics Dashboard

```

javascript

```

```

import { Chart } from 'chart.js';
import { Line, Bar, Pie } from 'react-chartjs-2';

// Jobs completed over time
<Line
  data={{
    labels: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
    datasets: [{
      label: 'Completed Jobs',
      data: [12, 19, 15, 25, 22, 30],
      borderColor: '#F59E0B',
    }],
  }}
/>

// Jobs by technician
<Bar
  data={{
    labels: technicians.map(t => t.name),
    datasets: [{
      label: 'Jobs Assigned',
      data: technicians.map(t => t.jobCount),
    }],
  }}
/>

```

3. AI-Powered Features

```

javascript

// Predictive maintenance (suggest when to schedule next service)
const suggestNextService = async (customerId) => {
  const history = await jobCardAPI.getAll({ customer_id: customerId });
  // ML model analyzes patterns
  // Returns suggested date
};

// Auto-fill job description based on title
const [title, setTitle] = useState("");
const [suggestedDescription, setSuggestedDescription] = useState("");

useEffect(() => {
  if (title.toLowerCase().includes('maintenance')) {
    setSuggestedDescription('Routine maintenance including cleaning, toner replacement, and calibration.');
```

4. Multi-Language Support (i18n)

```
javascript

import i18n from 'i18next';
import { useTranslation } from 'react-i18next';

i18n.init({
  resources: {
    en: {
      translation: {
        'jobs.pending': 'Pending',
        'jobs.completed': 'Completed',
      }
    },
    sw: {
      translation: {
        'jobs.pending': 'Inasubiri',
        'jobs.completed': 'Imekamilika',
      }
    },
  },
  lng: 'en',
});

// Usage
const { t } = useTranslation();
<StatusBadge label={t('jobs.pending')} />
```

14. Conclusion

14.1 Frontend Highlights

Architecture:

- ✓ Component-based architecture (reusable UI components)
- ✓ Context API for global state (auth + toasts)
- ✓ React Router for navigation (nested routes)
- ✓ Custom hooks for shared logic

User Experience:

- ✓ Role-specific interfaces (Supervisor vs Technician)
- ✓ Loading states (skeleton screens)

-  Empty states (helpful illustrations)
-  Error handling (toast notifications)
-  Responsive design (mobile-first)
-  Smooth animations (CSS transitions)

Code Quality:

-  Consistent naming conventions
-  Component composition (small, focused components)
-  TypeScript-ready (can migrate incrementally)
-  Separation of concerns (UI vs logic)

Performance:

-  Efficient re-renders (memoization where needed)
-  Lazy loading (planned for routes)
-  Optimized images (lazy loading)
-  Small bundle size (~45 KB gzipped)

14.2 What Makes This Production-Ready?

1. **Error Boundaries:** Graceful error handling (toast + error UI)
2. **Loading States:** Always show feedback (no hanging UI)
3. **Authentication:** Persistent sessions (localStorage)
4. **Responsive:** Works on mobile, tablet, desktop
5. **Accessibility:** Semantic HTML, keyboard navigation
6. **SEO-Ready:** Meta tags, proper routing
7. **Maintainable:** Clear structure, consistent patterns

14.3 Lessons Learned

What Went Well:

- Tailwind CSS made styling fast and consistent
- Context API was sufficient for this app size
- Component reusability (EmptyState, Skeleton, Toast)
- Mobile-first approach simplified responsive design

Challenges:

- Managing form state (many controlled inputs)
- Synchronizing UI state with server state

- Toast notification timing (auto-dismiss vs manual)

What I'd Do Differently:

- Use React Query for server state caching
 - Add TypeScript from the start
 - Implement error boundaries earlier
 - Use a form library (React Hook Form)
-

15. Presentation Tips

15.1 Demo Flow Recommendation

1. Start with Architecture (5 min)

- Show component tree
- Explain state management (Context API)
- Highlight routing structure

2. Live UI Demo (20 min)

- **Login** → Show different dashboards (supervisor vs technician)
- **Supervisor flow:**
 - View stats dashboard
 - Create job card
 - Filter job list
 - View job detail
- **Technician flow:**
 - View assigned jobs
 - Start a job
 - Complete a job
 - Show completion summary

3. Code Walkthrough (25 min)

- Show component structure (EmptyState, Skeleton, Toast)
- Explain authentication flow (AuthContext)
- Walk through API service layer
- Highlight responsive design (Tailwind)

4. Q&A Preparation (rest of time)

15.2 Common Questions & Answers

Q: Why not use Redux?

A: Context API is sufficient for this app's state needs. We only have:

- **Auth state:** User, token, login/logout
- **UI state:** Toasts
- **Server state:** Fetched on demand, not cached client-side

If the app grows, I'd add React Query for server state caching, not Redux.

Q: How do you handle stale data?

A: Currently, we refetch on every mount. For production, I'd add:

- **React Query:** Automatic refetching, background updates
- **Optimistic updates:** Update UI before server confirms
- **Cache invalidation:** Refresh data when related actions occur

Q: What about TypeScript?

A: Time constraint for this project. TypeScript would add:

- **Type safety:** Catch bugs at compile time
- **Better IDE support:** Autocomplete, refactoring
- **Documentation:** Types are self-documenting

Migration path: Add `tsconfig.json`, rename files `.tsx`, type gradually.

Q: How do you handle form validation?

A: Currently manual validation in components. For production, I'd use:

- **React Hook Form:** Less boilerplate, better performance
- **Zod:** Schema validation (similar to backend's Joi)

Q: Why Tailwind over CSS-in-JS?

A:

- **Performance:** No runtime overhead
- **Productivity:** Utility classes faster than writing custom CSS
- **Consistency:** Design system baked in (spacing, colors)
- **Bundle size:** Purge unused classes (small CSS file)

Q: How do you test this?

A: Testing strategy:

- **Unit tests:** Test pure components (Jest + React Testing Library)
- **Integration tests:** Test user flows (Cypress, Playwright)
- **Visual regression:** Snapshot testing (Percy, Chromatic)

Example:

```
javascript

test('shows empty state when no jobs', () => {
  render(<JobCardList jobs={} />);
  expect(screen.getByText('No job cards yet')).toBeInTheDocument();
});
```

Q: What about accessibility?

A: Current accessibility features:

- **Semantic HTML:** `<header>`, `<main>`, `<nav>`
- **Keyboard navigation:** All interactive elements focusable
- **Screen reader labels:** `aria-label` on icon buttons
- **Color contrast:** WCAG AA compliant

Future improvements:

- **Focus management:** Trap focus in modals
- **ARIA live regions:** Announce toast messages
- **Skip links:** Jump to main content

Q: How do you handle authentication expiry?

A: Current: Backend returns 401, frontend clears localStorage.

Improvement: Add token refresh:

```
javascript
```

```
// Intercept 401 responses
if (error.status === 401) {
  const newToken = await refreshToken();
  if (newToken) {
    // Retry original request
    return fetchWithAuth(url, options);
  } else {
    // Redirect to login
    logout();
  }
}
```

16. Appendix

16.1 File Structure Reference

```
src/
├── App.jsx           # Root component + routing
├── main.jsx          # Entry point (ReactDOM.render)
├── index.css         # Global styles (Tailwind imports)
├──
├── components/
│   ├── auth/
│   │   └── ProtectedRoute.jsx # Auth guard HOC
│   └── ui/
│       ├── EmptyState.jsx    # Empty state illustrations
│       ├── Logo.jsx          # Company logo
│       ├── Skeleton.jsx      # Loading skeletons
│       └── Toast.jsx          # Toast notifications
├──
├── context/
│   └── AuthContext.jsx       # Auth state + actions
├──
├── pages/
│   ├── auth/
│   │   ├── LoginPage.jsx     # Login form
│   │   └── RegisterPage.jsx  # Registration form
│   └── supervisor/
│       ├── SupervisorDashboard.jsx # Layout + nested routes
│       ├── StatsOverview.jsx      # Dashboard home
│       ├── AllJobsList.jsx        # Job cards table
│       ├── CreateJobCard.jsx      # Job creation form
│       ├── SupervisorJobDetail.jsx # Job view (no sig required)
│       └── CustomerList.jsx       # Customer CRUD
```

```
| | | — UserList.jsx      # User management
| | | — UserDetail.jsx    # User detail + stats
| | — technician/
| |   | — TechnicianDashboard.jsx # Mobile-optimized layout
| |   | — JobCardList.jsx      # Assigned jobs list
| |   | — JobDetail.jsx       # Job view + completion
| — services/
|   | — api.js              # API service layer (fetchWithAuth)
```

16.2 Tailwind Configuration

javascript

```
// tailwind.config.js
export default {
  content: [
    './index.html',
    './src/**/*..{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ['DM Sans', 'system-ui', 'sans-serif'],
        title: ['Syne', 'system-ui', 'sans-serif'],
      },
      colors: {
        amber: {
          400: '#FBBF24',
          500: '#F59E0B',
          600: '#D97706',
        },
      },
    },
  },
  plugins: [],
}
```

16.3 Package Dependencies

json

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.1",
    "autoprefixer": "^10.4.16",
    "postcss": "^8.4.32",
    "tailwindcss": "^3.4.0",
    "vite": "^5.0.8"
  }
}
```

16.4 Project Statistics

- **Lines of Code:** ~2,500 (frontend only)
- **Components:** 23
- **Pages:** 11
- **Context Providers:** 2 (Auth, Toast)
- **API Functions:** 15+
- **Routes:** 12

END OF FRONTEND DOCUMENTATION

This document serves as comprehensive technical documentation for the Job Card Management System frontend. It covers architecture, component design, state management, routing, UI/UX patterns, and deployment considerations suitable for presentation to senior software engineers.